

CockroachDB

Day 4: Advanced Features & Real-World Integration

 **Distributed SQL** — in the shape of a real system

CDC · Application Patterns · Migration · Kubernetes · Cost



Day 4 Schedule

Module 4: Advanced Features & Integration

1. Change Data Capture in Depth

 Lab 13 — CDC → Kafka → consumer with a resolved-timestamp frontier (*75 min*)

2. Application Integration: outbox, idempotency, pooling, ORMs

 Lab 14 — Outbox pattern + idempotent retry loop (*75 min*)

3. Schema Migrations, CI/CD & Migrating from PostgreSQL

 Lab 15 — Migrate a PostgreSQL schema and live data with MOLT (*90 min*)

4. Kubernetes Deployment



Section 1: Change Data Capture in Depth

Getting data out, correctly

Two Flavours

Core

```
EXPERIMENTAL CHANGEFEED  
FOR shop.orders;
```

- Streams to the SQL session
- No license, no sink
- Dies with the session
- Debugging only

Enterprise

```
CREATE CHANGEFEED  
FOR TABLE shop.orders  
INTO 'kafka://...'  
WITH updated, resolved='10s';
```

- Runs as a resumable job
- At-least-once delivery
- Survives restarts and rebalances
- Everything in production

Sinks: Kafka, webhook, cloud storage (S3/GCS/Azure), Google Pub/Sub.

Envelope Options and What They Cost

Option	Adds	Costs
updated	MVCC timestamp of the change	Negligible
resolved = 'Ns'	Frontier watermarks	More messages
diff	The before image	~2× payload
key_in_value	PK inside the value	Small
format = avro	Avro + schema registry	Needs a registry
mvcc_timestamp	Raw MVCC even for deletes	Negligible

```
{
  "after": { "id": "...", "status": "paid" },
  "before": { "id": "...", "status": "new" },
  "key": [ "..."],
  "updated": "1716400000000000.000000000"
}
```

The Resolved-Timestamp Frontier

This is the part everyone gets wrong. Changefeed messages arrive *out of order across ranges*. A row you receive at 14:05 may have a timestamp of 14:02, and a 14:01 row may still be in flight.

A `resolved` message is the cluster promising: *"no further message will arrive with a timestamp below this value."*

Question

Answer

At-Least-Once Means Duplicates Are Normal

After a job restart, a lease transfer, or a rebalance, messages are re-emitted. Your consumer must be idempotent — three ways:

Technique	How	Best for
Upsert by PK	<code>INSERT ... ON CONFLICT DO UPDATE</code>	Materialized copies
Timestamp guard	Apply only if <code>updated > last_seen</code>	Out-of-order safety
Dedup ledger	Store <code>(key, ts)</code> seen-set with TTL	Side effects

For side effects — emails, charges, notifications — the ledger is the only safe option. You cannot "re-send" an email idempotently by upserting a row.

Schema Changes Under an Active Feed

<code>schema_change_policy</code>	Behaviour
<code>backfill</code> (default)	Re-emit the whole table after the change
<code>nobackfill</code>	Continue without re-emitting existing rows
<code>stop</code>	Fail the job so a human decides

The contract rule: additive changes (a new nullable column) are safe. Renames and type changes are not. Version your consumer's schema handling the same way you version an API.

The hidden cost of a paused feed: its protected timestamp stops MVCC garbage collection.



Lab 13: CDC → Kafka → Frontier Consumer



75 min

Objectives

- Run Core and Enterprise changefeeds; decode the envelopes
- Build a consumer that buffers until `resolved` and applies in order
- Prove idempotency by replaying from an earlier cursor
- Handle a schema change under an active feed
- Kill the sink and confirm nothing was lost



Setup

```
docker compose up -d # single-broker Kafka  
scripts/crdb sql -e "SET CLUSTER SETTING kv.rangefeed.enabled = true;"
```

Deliverable

A working frontier consumer plus a failure drill: broker killed, feed recovered, zero events lost, lag measured.



Section 2: Application Integration Patterns

Retries, outbox, idempotency, pooling

SQLSTATE 40001 Is Not an Error

Under `SERIALIZABLE`, a transaction that would violate serializability is aborted and must be retried by the client. That is the concurrency-control protocol, not a failure.

```
for attempt in range(1, MAX_ATTEMPTS + 1):
    try:
        with conn:
            with conn.cursor() as cur:
                body(cur)
            # the ENTIRE transaction, re-executed
        break
    except psycopg2.errors.SerializationFailure:
        if attempt == MAX_ATTEMPTS: raise
        time.sleep(random.uniform(0, min(0.5, 0.005 * 2**attempt))) # jitter!
```

Two mistakes to avoid: retrying only the failing *statement* (the values you read are stale), and backoff *without jitter* (all your clients collide again at exactly the same moment).

Before You Write a Retry Loop

Read-modify-write

```
BEGIN;  
SELECT balance FROM accounts  
  WHERE id = $1;  
-- app computes bal + 1  
UPDATE accounts SET balance = $2  
  WHERE id = $1;  
COMMIT;
```



One statement

```
UPDATE accounts  
  SET balance = balance + 1  
  WHERE id = $1;
```

Atomic server-side. No retry loop needed. No round trip in the middle of the transaction.

Use the driver's helper rather than hand-rolling: `crdbpgx.ExecuteTx` (Go), `run_transaction()` (SQLAlchemy).

The Outbox Pattern

The dual-write problem: your service writes an order to the database *and* publishes an event to Kafka. If the process dies between them, the two systems disagree forever. No amount of retry logic fixes this.

```
BEGIN;  
  INSERT INTO orders (account, amount) VALUES ($1, $2) RETURNING id;  
  INSERT INTO events_outbox (topic, aggregate, payload)  
    VALUES ('order.created', $order_id, $payload);  
COMMIT;  
-- CDC ships events_outbox downstream. One transaction, one truth.
```

Designing the Outbox Table

```
CREATE TABLE events_outbox (  
  id          UUID NOT NULL DEFAULT gen_random_uuid(),  
  created     TIMESTAMPTZ NOT NULL DEFAULT now(),  
  topic       STRING NOT NULL,  
  aggregate   UUID NOT NULL,  
  payload     JSONB NOT NULL,  
  PRIMARY KEY (id)  
) WITH (ttl_expire_after = '7 days', ttl_job_cron = '@hourly');
```

Decision	Why
UUID primary key	Append-only at high rate — a sequential key hotspots the rightmost range
Row-level TTL	The outbox is a buffer, not a log. Without TTL it grows forever
No status column, no UPDATE	A polled WHERE status='pending' queue serializes on one range

The anti-pattern it replaces: `SELECT ... WHERE status='pending' FOR UPDATE LIMIT 100` then `UPDATE ... SET status='sent'`. Every worker contends on the same rows; throughput caps out no matter how many workers you add.

Idempotency Keys

```
CREATE TABLE idempotency (  
  key      STRING PRIMARY KEY,  -- client-supplied, one per user action  
  account  UUID NOT NULL,  
  response JSONB NOT NULL,      -- what we returned the first time  
  created  TIMESTAMPTZ NOT NULL DEFAULT now()  
) WITH (ttl_expire_after = '24 hours');
```

Claim the key in the same transaction as the work. Claim it in a separate transaction and a crash between the two leaves the key claimed and the work undone — every retry then returns "already processed" for work that never happened.

A concurrent duplicate loses the race on the primary key and rolls back its whole transaction, including the order. Exactly one order exists.

Optimistic vs Pessimistic Concurrency

OCC (version column)

```
UPDATE accounts
SET balance = $1, version = version + 1
WHERE id = $2 AND version = $3;
-- 0 rows => someone won; re-read
```

No locks. Best at low contention and when there's think-time between read and write.

SELECT FOR UPDATE

```
BEGIN;
SELECT balance FROM accounts
WHERE id = $1 FOR UPDATE;
UPDATE accounts SET balance = $2
WHERE id = $1;
COMMIT;
```

Writers queue instead of racing. Best at high contention on a known row.

Lab 14 benchmarks both. Measure your actual contention before choosing — the "obvious" answer flips depending on it.

Connection Pooling

Setting	Rule
Per app instance	<code>(cores × 2) + spindles</code> , validated against the measured knee
Cluster total	Under ~4× total vCPU across all nodes
PgBouncer mode	transaction pooling (session pooling wastes connections)
Timeouts	Set <code>statement_timeout</code> and pool acquisition timeouts

An oversized pool adds no throughput. It moves the queue from your app into the database, where it's harder to see and more expensive to drain. Little's Law: `concurrency = throughput × latency`. 2,000 QPS at 8 ms is *16* connections, not 200.

Transaction pooling breaks session-scoped features — use `SET LOCAL`, not `SET`.

Driver & ORM Landscape

Language	Driver	ORM	Retry helper
Python	psycopg / psycopg2	SQLAlchemy, Django	<code>sqlalchemy-cockroachdb</code>
Go	pgx	GORM, sqlx, ent	<code>crdbpgx.ExecuteTx</code>
Node.js	node-postgres	Prisma, Knex, TypeORM	Hand-rolled or <code>cockroach-node</code>
Java/Kotlin	pgjdbc	Hibernate (CRDB dialect), jOOQ	Retry aspect / interceptor
Ruby	pg	ActiveRecord (adapter)	<code>activerecord-cockroachdb</code>

The ORM checklist: does it retry 40001? does it emit `SERIAL` primary keys by default (change them)? does it wrap every request in one transaction? does it do N+1 queries — each of which is a distributed read?



Lab 14: Outbox Pattern + Idempotent Retry Loop



75 min

Objectives

- Measure a naive service vs one with a correct retry loop
- Design an outbox table that avoids a hot range and a polled queue
- Prove atomicity with a simulated crash before commit
- Defend idempotency against sequential *and* concurrent duplicates
- Benchmark OCC against `SELECT FOR UPDATE`



The measurement that lands

40 concurrent read-modify-writes with no retry loop drop a meaningful share of requests. The same code with jittered retries loses none — and the single-statement version needs no retry at all.



Section 3: Migrations, CI/CD & Moving off PostgreSQL

The one chance to fix your primary keys for free

Online Schema Changes in CI/CD

Change	Runs as	Deploy impact
CREATE INDEX	Background job	None — writes continue
ADD COLUMN with default	Background job	None
DROP COLUMN	Background job	None (data removed async)
ALTER COLUMN TYPE	Validating backfill	Schedule it
ADD CONSTRAINT ... NOT NULL	Full validation scan	Schedule it
ADD FOREIGN KEY	Full validation scan	Schedule it

The rolling-deploy rule: a migration must be compatible with both the old and the new application version, because both run at once. Add-then-backfill-then-switch-then-drop, across three deploys — never rename in one step.

PostgreSQL → CockroachDB: The Cheat Sheet

PostgreSQL	CockroachDB	Action
SERIAL / BIGSERIAL	Works, means <code>unique_rowid()</code>	Change to UUID
Sequences	Supported, serialize on one range	Avoid on hot paths
plpgsql triggers	Not supported	App code, or a UDF
Stored procedures	UDFs only	Rewrite
TIMESTAMP	Prefer <code>TIMESTAMPTZ</code>	Change
JSON	Stored as <code>JSONB</code>	Fine — inverted-index it
Table inheritance	Not supported	Redesign
PG extensions	Not supported	Find the built-in
Foreign keys	Supported; each check is a distributed read	Audit hot paths

```
# Ask the source what it uses BEFORE you promise a timeline
psql -c "SELECT count(*) FROM information_schema.triggers;"
psql -c "SELECT extname FROM pg_extension;"
psql -c "SELECT sequencename FROM pg_sequences;"
```

The Highest-Value Hour of Any Migration

Changing a primary key *after* cutover means another migration. Changing it *during* the migration costs nothing — the data is already being rewritten.

Legacy design	Problem	Playbook fix
<code>id SERIAL PRIMARY KEY</code>	Rightmost-range hotspot	#3 Per-Tenant Co-located PK
<code>BIGSERIAL event log</code>	Pure append at max rate	#1 Hash-Sharded Time-Series PK
Index on <code>status</code>	Low cardinality, rarely selective	#6 Hot/Cold partial index
Nothing ages out events	Unbounded growth	#9 TTL table

Zero-Downtime Cutover — Name the Rollback

Phase 1	Schema on CRDB, PG live	← rollback: drop CRDB
Phase 2	Bulk load + continuous replication	← rollback: stop replication
Phase 3	Dual write, PG authoritative	← rollback: stop CRDB writes
Phase 4	Shadow reads, compare, serve PG	← rollback: stop shadow reads
Phase 5	Flip: CRDB authoritative, PG still written	← rollback: flip back (minutes)
Phase 6	Stop writing PG, decommission	← rollback: restore from backup

The flip gate — evidence, not a date

- Replication lag < 1 s sustained for 24 h
- Shadow-read divergence = 0 over 24 h
- Every query at or under its PostgreSQL latency
- A tested rollback with a measured RTO
- Backups running on the target, restore drill completed

Migrations Regress Queries — Find Out First

Symptom	Cause	Fix
Full scan where PG used an index	Missing/stale statistics	<code>CREATE STATISTICS</code>
Slow multi-row <code>IN</code> lookup	Keys scattered across ranges	Reorder the PK to co-locate
Slow <code>COUNT (*)</code>	Distributed scan, no shortcut	Follower read, or a maintained counter
Slow FK-heavy inserts	Each FK check is a distributed read	Batch; drop non-critical FKs
High p99, fine p50	Contention or cross-range txns	Day 2 techniques

Always **CREATE STATISTICS** before benchmarking a migration. A freshly loaded table has none, and you'd be measuring the optimizer flying blind.

When NOT to Migrate

✓ CockroachDB earns it

- Survivability requirements (zone/region)
- Write throughput beyond one machine
- Data locality / residency rules
- Multi-region low-latency reads with strong consistency
- No tolerance for failover windows

✗ Stay on PostgreSQL

- Single region, one instance is fine
- Heavy analytical scans and window functions
- Deep extension dependence (PostGIS specifics)
- Small team, no distributed-systems experience
- The pain you have isn't one of the ✓ list

Being able to argue the ✗ case credibly is what makes the ✓ case believable.



Lab 15: Migrate a PostgreSQL Schema and Live Data



90 min

Objectives

- Audit a legacy schema and find every incompatibility before migrating
- Redesign the primary keys using four Playbook patterns
- Move data with MOLT Fetch; verify with counts *and* a business checksum
- Add constraints after the load and measure the validation cost
- Compare plans on both engines and fix the regressions



Setup

```
docker run -d --name lab15-pg -p 5432:5432 \
  -e POSTGRES_PASSWORD=pg -e POSTGRES_DB=legacy postgres:16
molt fetch --source "$PG" --target "$CRDB" --direct-copy
molt verify --source "$PG" --target "$CRDB"
```



Section 4: Kubernetes Deployment

Stateful, and the state matters

Kubernetes Object → CockroachDB Concept

Object	Role
StatefulSet	Stable identities (<code>crdb-0</code> , <code>crdb-1</code> ...) — a node's identity must survive restarts
Headless service	Peer discovery for <code>--join</code>
Public service	Client entry point
PVC per pod	The store. Deleting it deletes that node's data
PodDisruptionBudget	Stops <code>kubectl drain</code> from taking quorum with it
topologySpreadConstraints	Keeps replicas out of the same failure domain

```
kubectl patch crdbcluster crdb --type=merge -p '{"spec":{"nodes":5}}'
kubectl patch crdbcluster crdb --type=merge -p '{"spec":{"cockroachDBVersion":"v23.2.6"}}'
```

The Two Mistakes That Cause Outages

1. No `--locality`

Kubernetes knows the pods are in three zones. CockroachDB does not, unless you tell it. Without `--locality`, the allocator can put all three replicas of a range in one zone — and you find out when that zone fails.

2. Scaling down before decommissioning

```
# WRONG: removes the pod, replicas go offline instantly
kubectl patch crdbcluster crdb -p '{"spec":{"nodes":3}}'

# RIGHT: move replicas off while the node still serves, THEN remove it
kubectl exec cockroachdb-client-secure -- ./cockroach node decommission 5 \
  --wait=all --certs-dir=/cockroach/cockroach-certs --host=crdb-public
kubectl patch crdbcluster crdb -p '{"spec":{"nodes":4}}'
```

Operator vs Helm vs Manual

Approach	Handles for you	You still own
cockroach-operator	Certs, rolling upgrades, scaling, decommission ordering	Locality, storage class, monitoring
Helm chart	Templating, values-driven config	Upgrade ordering, cert lifecycle, decommission
Manual StatefulSet	Nothing	Everything, including the two mistakes above

Production checklist

- `--locality` reflects real failure domains
- Topology spread / anti-affinity prevents co-location
- PodDisruptionBudget protects quorum
- SSD storage class, `WaitForFirstConsumer`
- `Requests == limits` for guaranteed QoS; memory limits set
- ServiceMonitor scraping / `status/vars`



Lab 16: CockroachDB on Kubernetes with kind



90 min

Objectives

- Deploy via the operator with TLS and topology spread
- Scale out and roll a version upgrade under live load
- Kill a pod and a node; watch identity and PVC reattachment
- Decommission in the correct order
- Compare operator / Helm / manual with reasons



Setup

```
kind create cluster --name lab16 --config kind-config.yaml # 3 zoned workers
kubectl apply -f ../install/crds.yaml
kubectl apply -f ../install/operator.yaml
kubectl apply -f crdb.yaml
```



Section 5: Advanced Features, Cost & Wrap-Up

Advanced Features Survey

Feature	Use it for	Watch out for
Row-level TTL	Sessions, tokens, outbox, event logs	Cron frequency vs foreground load
UDFs	Replacing simple triggers/procedures	Not a general procedural language
Spatial types	Geo queries, PostGIS-style workloads	Function coverage differs from PostGIS
Vector indexes	Embeddings, similarity search	Check your version — recent feature
Full-text search	Basic text search without a separate engine	Not a replacement for a search engine
Follower reads	Stale-tolerant reads at local latency	Bounded staleness is a product decision
GLOBAL tables	Read-anywhere reference data	Writes are slow by design

Version matters for several of these — check the release notes for the version you actually run, not the newest one.

Deployment Models & Cost

Model	Best for	You operate	Cost shape
Serverless / Basic	Dev, small apps, spiky traffic	Nothing	Per request + storage
Dedicated / Standard	Production without an ops team	Schema and app	Per node-hour
Self-hosted	On-prem, regulated, full control	Everything	Hardware + staff

The honest comparison includes the ops salary line. A self-hosted cluster is cheaper on the invoice and more expensive on the payroll. Estimate both before deciding.

Enterprise vs Core changes what's available around the edges (some backup, changefeed, and multi-region features) — check the current matrix for your version rather than trusting a slide.

The Anti-Pattern Checklist — Take This to Code Review

Spot this	Say this
<code>id SERIAL PRIMARY KEY</code> on a hot table	UUID, or hash-shard it
<code>UPDATE counters SET n = n + 1 WHERE id = ...</code>	Sharded counter
Low-cardinality lead column in the PK	Reorder or rethink
"Let's index every column just in case"	Each index is 3 more physical writes
<code>DELETE FROM logs WHERE created < ...</code> on a cron	Row-level TTL
App writes to Kafka <i>and</i> the database	Outbox + CDC
Polled queue with <code>status='pending'</code>	CDC off the write path
No 40001 retry loop	It will drop requests under load
Pool of 200 per app instance	Size it from the measured knee



Four Days, One Idea

In a distributed SQL database, the schema is the performance model, the capacity model, and the availability model.

- Day 1 — the primary key is a distribution decision, not a uniqueness decision
- Day 2 — contention, plans, and throughput all trace back to that decision
- Day 3 — so do your metrics, your node count, and your hardware bill
- Day 4 — and your integration patterns, your migration, and your Kubernetes topology

Take home: the [Schema Patterns Playbook](#) — ten patterns, each with the tran it avoids and the measurement that proves it

Where to Go Next

- Docs: cockroachlabs.com/docs — the performance and multi-region sections repay a slow read
- Forum: forum.cockroachlabs.com
- Release notes: read them before upgrading; behaviour changes are documented there
- Practice: re-run Lab 8's PK bake-off and Lab 10's sizing exercise against *your* schema

Your first week back

1. Run the anti-pattern checklist over one real schema
2. Measure the batch-size knee and concurrency knee for one real workload
3. Do one DR drill and record the measured RTO



Thank you — go break something on purpose.